Informed Search

Using heuristics to search smarter

Greedy, A*, and Adversarial Search

Uninformed vs. Informed Search

Uninformed Search — Reminder

No heuristic: the algorithm does not estimate how close a node is to the goal.

Examples:
- DFS — go deep first
- BFS — go level by level
- UCS — choose lowest g(n) cost so far

Informed Search — What Changes?

Adds a heuristic h(n): an estimate of remaining cost to the goal.

Examples:
- Greedy — uses h(n)
- A* — uses g(n) + h(n)

Main difference: informed search has guidance about the goal.

Remember: Uninformed = no h(n) | Informed = uses h(n)

Heuristic Function

Core Concept

A heuristic estimates how close a state is to the goal.

$h(n)$ = estimated cost from n to the goal

Common Examples

- Straight-line distance
- Manhattan distance
- Misplaced tiles
- Estimated remaining travel time

Remember A heuristic is an estimate. It can guide the search, but it can also be wrong.

QUICK CHECK

Which one is a heuristic for a navigation problem?

A. The current city

B. The exact shortest path already computed

C. The straight-line distance to the destination

D. The number of roads in the whole map

Correct Answer

C

The straight-line distance to the destination

Straight-line distance is not the exact road distance, but it is a useful estimate of how close a city is to the destination.

Greedy Best-First Search

Priority: f(n) = h(n)

Greedy asks

“Which option looks closest to the goal right now?”

It always expands the node with the smallest h(n).

Important Limitation

Greedy does not consider the cost already paid to reach the node.

So it can be fast, but it is not guaranteed to find the cheapest path.

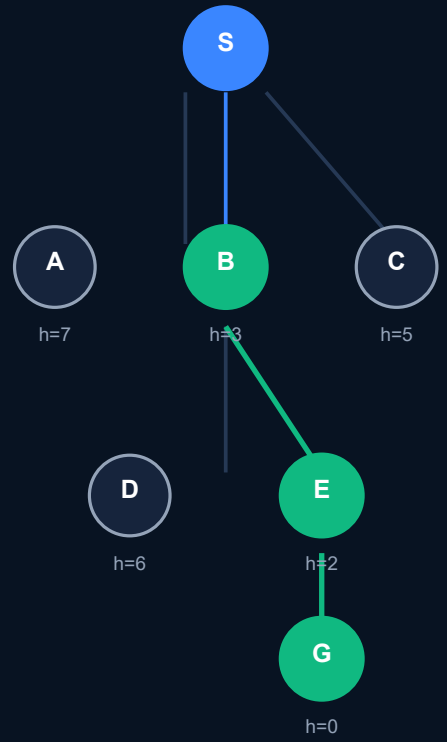
Greedy Search: Simple Example

Start: S | Goal: G

Greedy always chooses the lowest h(n):

1. From S → choose B (h=3)
2. From B → choose E (h=2)
3. From E → choose G (h=0)

Path: S → B → E → G



Greedy follows the heuristic only.

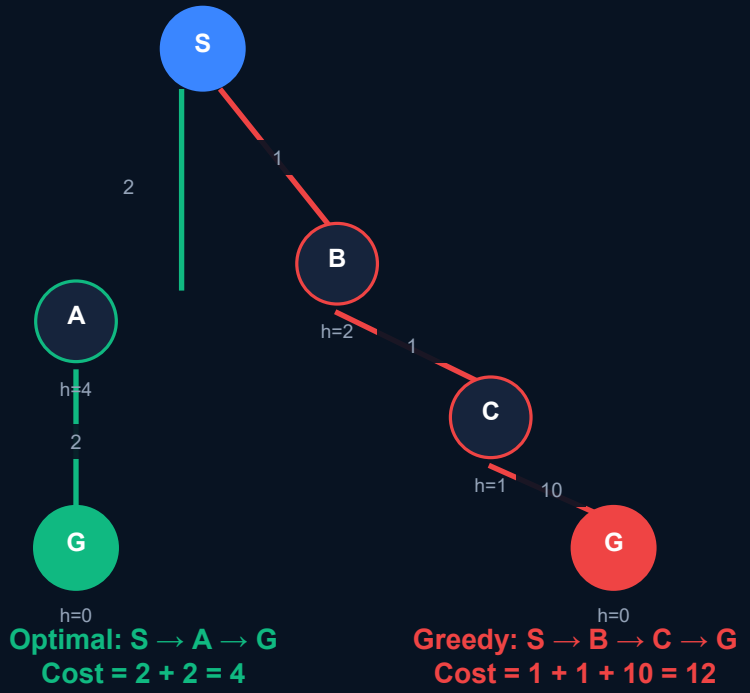
Greedy Search Can Be Misled

The Pitfall

Greedy chooses what looks closest to the goal.

It may ignore an expensive path that appears attractive.

Source example:
- Greedy path cost = 12
- Better path cost = 4



INFORMED SEARCH

A* Search Algorithm

g(n)

Actual cost already paid from the start to node n.

h(n)

Estimated cost from node n to the goal.

A* combines both

It considers the past and the estimated future.

$$f(n) = g(n) + h(n)$$

A* asks: “Which path looks cheapest overall?”

UCS vs. Greedy vs. A*

Algorithm	Priority	Simple meaning	Optimal?
UCS	g(n)	Cheapest cost so far	Yes*
Greedy	h(n)	Looks closest to goal	Not guaranteed
A*	g(n)+h(n)	Cheapest estimated total	Yes**

Fun fact: if h(n) = 0 everywhere, A* behaves like UCS.

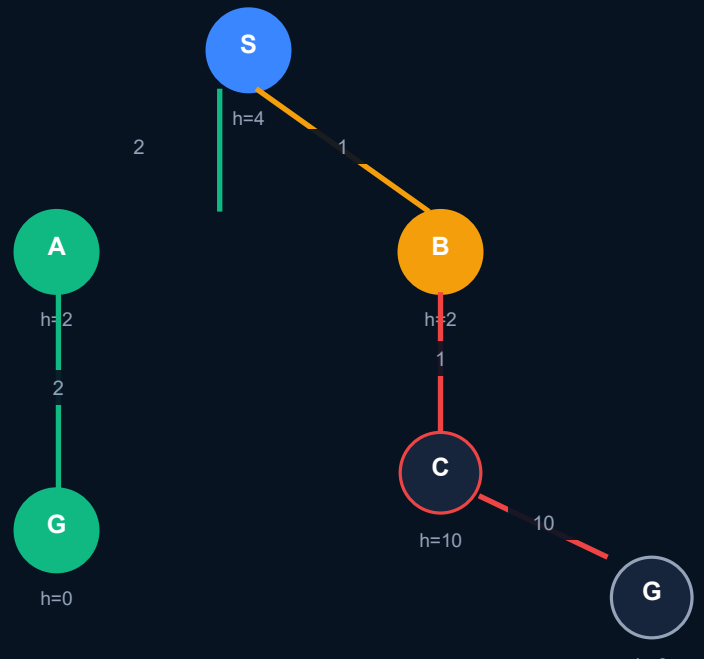
A*: Simple Example

Heuristics

h(S)=4 h(A)=2
h(B)=2 h(C)=10
h(G)=0

What happens?

1. A* first tries B: f=1+2=3
2. From B, C becomes f=2+10=12
3. A is still f=2+2=4, so A* returns to A
4. A → G gives total cost 4



A* corrects the tempting first choice because it compares total estimated cost.

QUICK CHECK

A* uses which priority formula?

A. f(n) = h(n)

B. f(n) = g(n)

C. f(n) = g(n) + h(n)

D. f(n) = g(n) - h(n)

Correct Answer

C **f(n) = g(n) + h(n)**

A* combines the cost already paid with the estimated cost that remains.

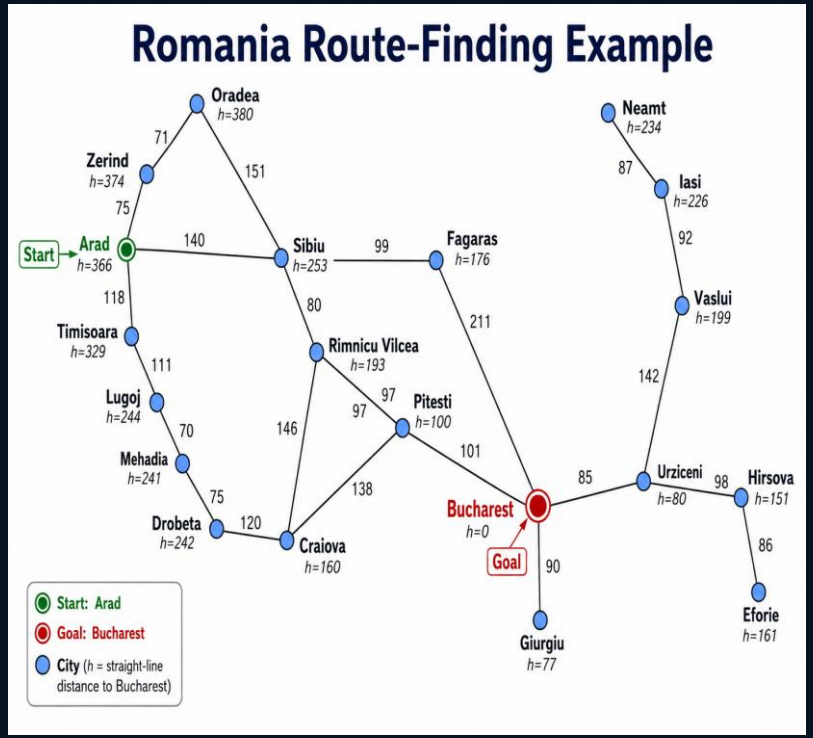
Romania Route-Finding Example

Problem Setup

Goal: find a route from Arad to Bucharest.

Use:
- g(n) = road distance traveled
- h(n) = straight-line distance to Bucharest

Compare:
- Greedy Search
- A* Search



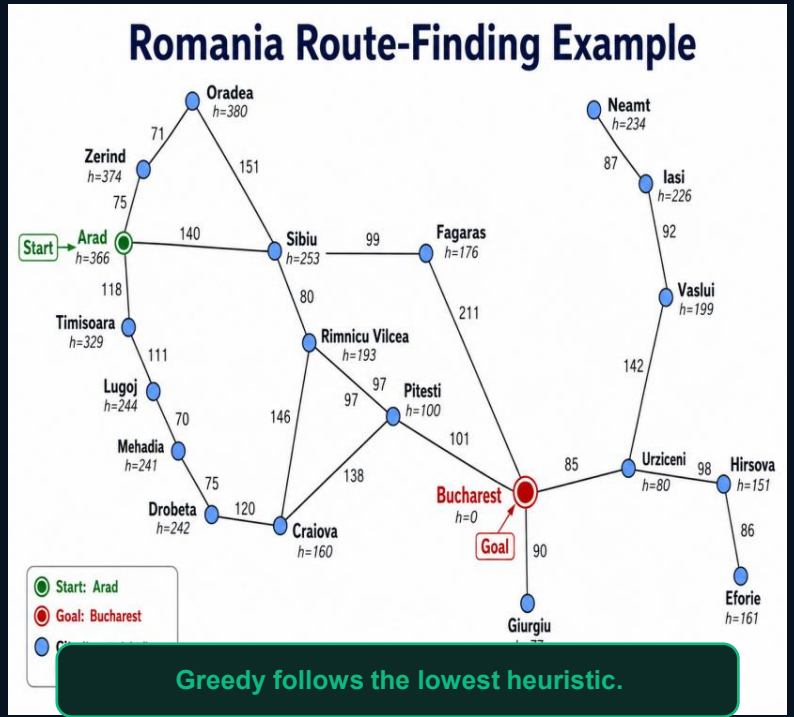
Romania Example: Greedy Search

Greedy decisions

1. Arad → Sibiu
2. Sibiu → Fagaras
3. Fagaras → Bucharest

Greedy Path:
Arad → Sibiu → Fagaras → Bucharest

Total cost = 450



Romania Example: A* Search

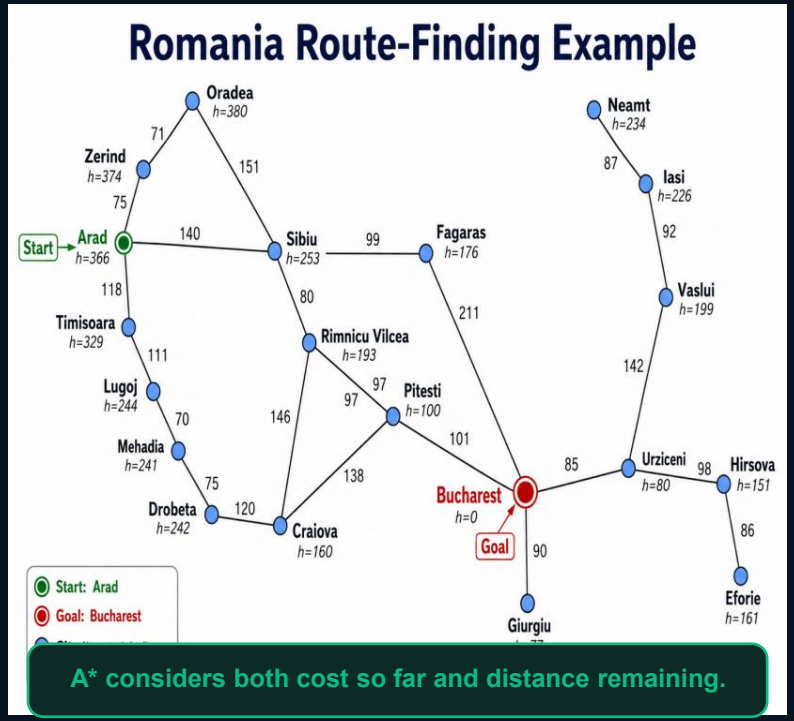
A* balances g + h

A* does not simply choose the city that looks closest.

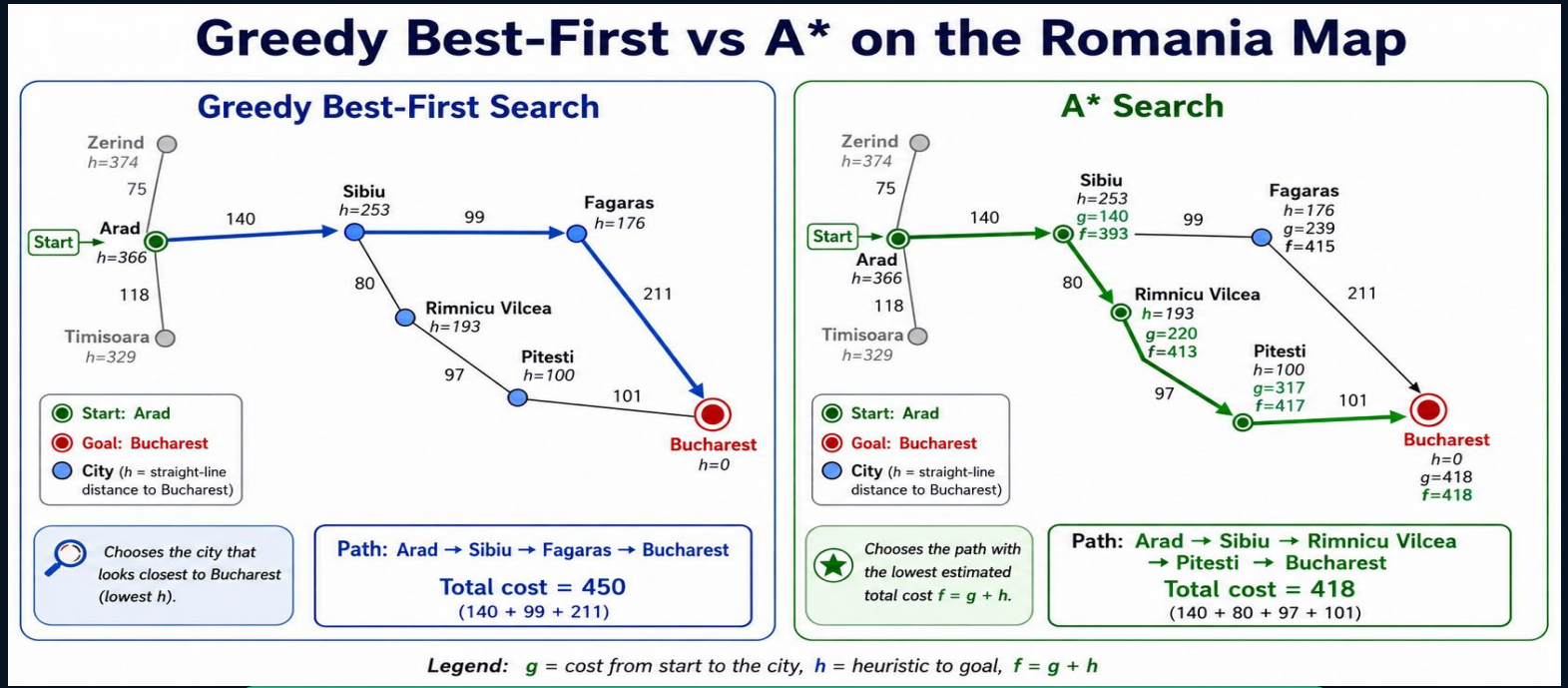
Final chosen path:
Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest

Total cost = 418

This is cheaper than Greedy's 450.



Greedy vs. A* on the Romania Map



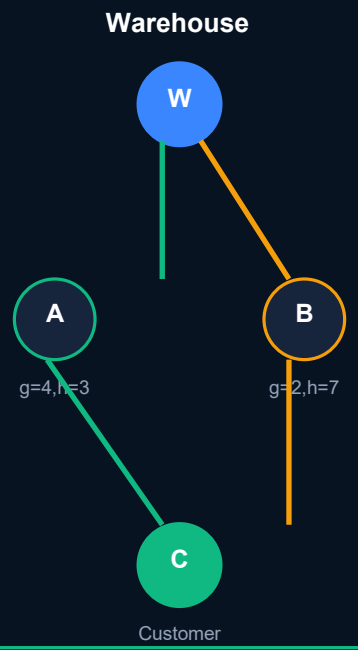
Greedy: 450 | A*: 418 → A* finds the lower-cost route

COMPANY APPLICATION

Practical Application: Delivery Company

Business Problem

A delivery company must choose a good route from a warehouse to a customer without exploring every possible road.



A*: compare g(n) + h(n)

How A* Helps

- g(n): time/distance already traveled
- h(n): estimated remaining time/distance
- Choose the route with the best total estimate

Business value:
Faster route planning and less wasted travel.

COMPANY APPLICATION

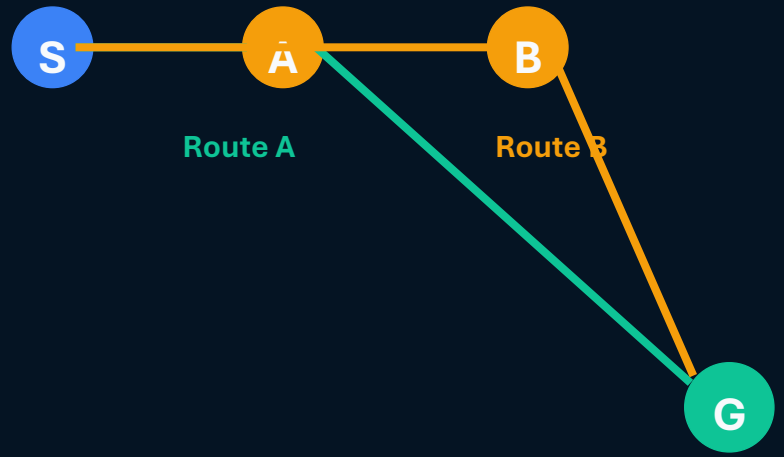
Google Maps: Choosing a Promising Route

Problem

Many roads connect start and goal. Find a good route without exploring every road.

Search model

g(n): cost already traveled
h(n): estimated remaining time/distance
A*: choose the smallest g(n) + h(n)



Which route looks best when current cost + remaining estimate are considered?

Company connection: Google Maps publicly describes using traffic information and routing to determine useful routes. Teaching mapping to A* is simplified.

COMPANY APPLICATION

Amazon Robotics: Warehouse Path Planning

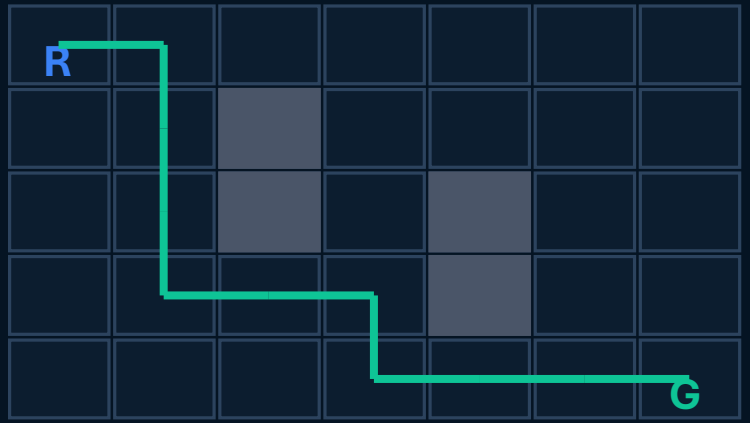
Problem

Robots share warehouse aisles. Some paths may be blocked or congested.

Search model

g(n): movement cost so far
h(n): estimated cost to the target
Use h(n) to focus on promising paths.

Warehouse Grid



Heuristic idea: “Which free cell appears to move me closer to the target?”

Company connection: Amazon Science describes centralized robot path planning and research on congestion-aware planning in fulfillment centers.

COMPANY APPLICATION

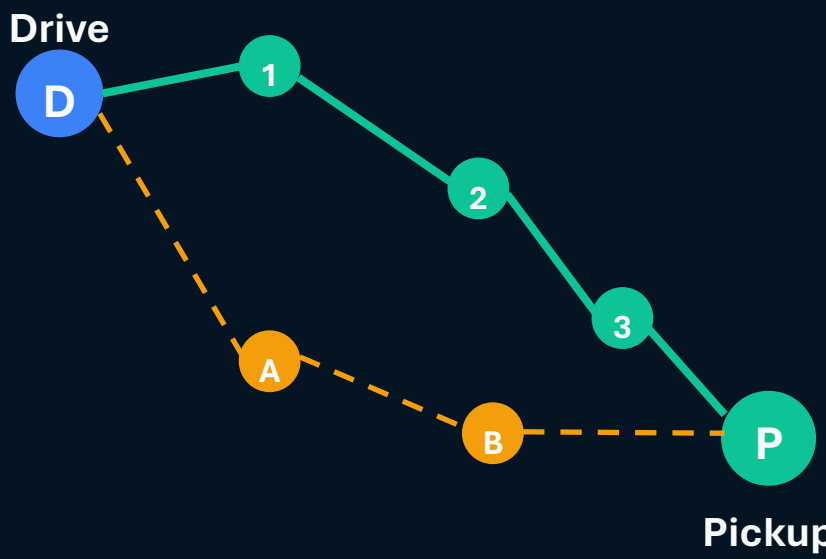
Uber: Routing and ETA Decisions

Problem

Find a useful route from driver to pickup while considering map and traffic information.

Search model

g(n): travel time so far
h(n): estimated remaining time
A* balances what is already paid with what remains.



Informed search can prioritize the route with the better estimated total travel cost.

Company connection: Uber Engineering describes routing engines that use map data and traffic measurements to find routes and estimate arrival times.

QUICK CHECK

Company Example MCQ 1

In the Google Maps teaching example, which value is the best heuristic h(n)?

A. Distance already traveled

B. Estimated remaining travel time to the destination

C. Number of roads in the entire city

D. The starting location

Correct: B — a heuristic estimates the remaining cost to the goal.

QUICK CHECK

Company Example MCQ 2

A warehouse robot uses A*. What does A* compare?

A. Only the distance already traveled

B. Only the estimated remaining distance

C. g(n) + h(n)

D. The number of nearby robots only

Correct: C — A* combines actual cost so far with estimated remaining cost.

QUICK CHECK

Review MCQ 3

Which statement best distinguishes informed from uninformed search?

A. Informed search always uses a stack

B. Uninformed search always finds a worse path

C. Informed search uses a heuristic to guide exploration

D. Uninformed search cannot use path cost

Correct: C — the defining difference is heuristic guidance h(n).

Adversarial Search

Planning against an active opponent

Game-playing search

What Makes Search Adversarial?

Standard Search

The environment does not deliberately try to block the agent.

The agent searches for a path to the goal.

Adversarial Search

Another agent actively chooses actions that can make our outcome worse.

We must plan for the opponent’s response.

Examples: Tic-Tac-Toe • Chess • Checkers • Connect Four

From a Plan to a Strategy

Ordinary Search: One Plan

Start → Action 1 → Action 2 → Goal

This works when the agent controls the actions and the environment is predictable.

Adversarial Search: A Strategy

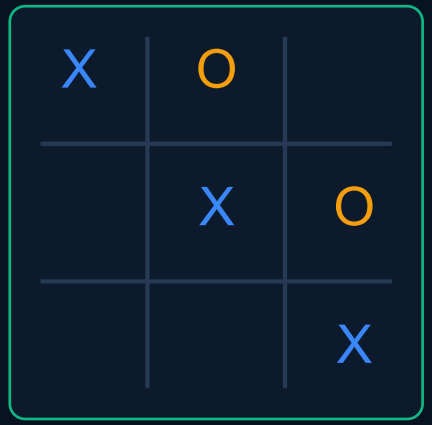
“If the opponent does X, I will respond with Y.”

The agent must consider several possible opponent responses.

The difference: a plan is one path; a strategy includes responses.

Formulating a Game: Tic-Tac-Toe

Component	Tic-Tac-Toe
State	Current 3 × 3 board
Player	X or O
Actions	Place a mark in an empty cell
Result	Board after the move
Terminal	X wins, O wins, or board is full
Utility	X wins: +1 | Draw: 0 | O wins: −1



Goal test becomes a terminal test + utility value.

Minimax

What Minimax Assumes

- Two players
- MAX tries to increase utility
- MIN tries to decrease utility
- Both choose their best moves

Minimax asks:

"If I make this move, what is the best response my opponent can make?"

Then choose the move with the best guaranteed result.

MAX → highest utility

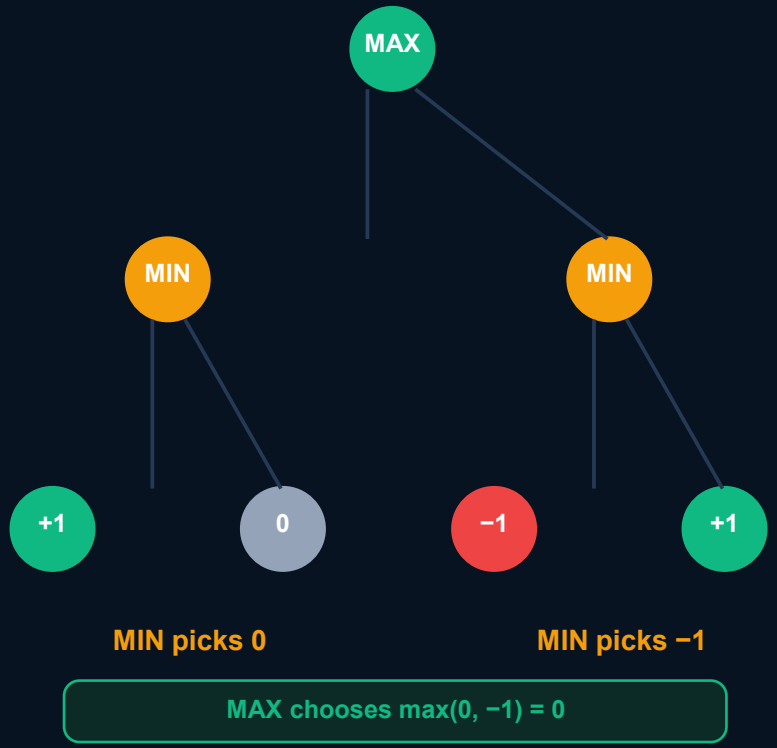
MIN → lowest utility

Minimax: Simple Example

Utility is from MAX's perspective:

+1 = MAX wins
 0 = draw
−1 = MAX loses

MIN chooses the smaller value.
MAX chooses the larger result.

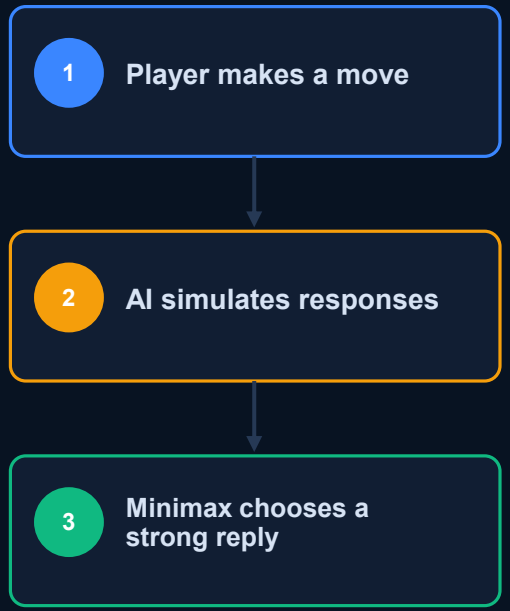


COMPANY APPLICATION

Practical Application: Game Company

Business Goal

A game company wants the computer-controlled opponent to make intelligent moves instead of choosing randomly.



Why It Matters

- Stronger computer opponents
- More consistent game difficulty
- Better player experience

Simple examples:
Tic-Tac-Toe and other turn-based strategy games.

QUICK CHECK

In Minimax, what does the MIN player try to do?

A. Choose the highest utility value

B. Choose the lowest utility value

C. Choose the smallest heuristic h(n)

D. Choose the shortest path

Correct Answer

B

Choose the lowest utility value

MIN represents the opponent trying to make MAX's outcome as small as possible.